Transaction and Concurrency Control

Chapter 10

Transaction

A transaction is a unit of program execution that accesses and possibly updates various data items.

A transaction is a set of changes that must all be made together.

It is a unit of program execution that access and possibly updates various data items.

Moreover, it is a program unit whose execution mayor may not change the contents of a database.

If the database was in consistent state before a transaction, then after execution of the transaction also, the database must be in a consistate.

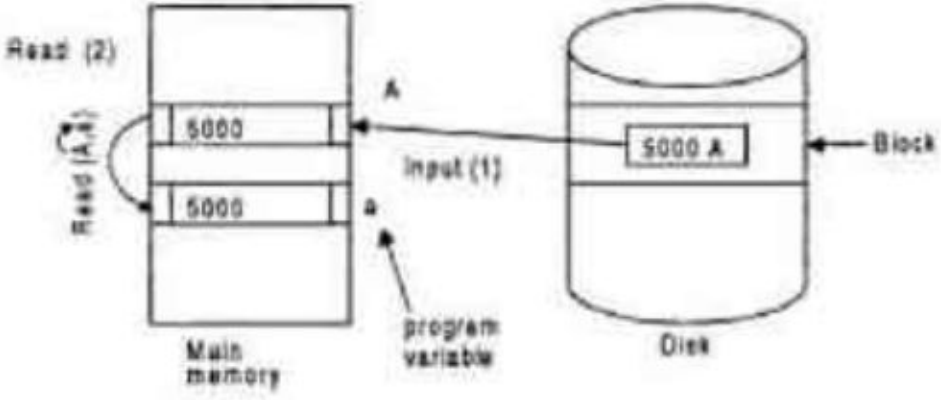
For example, a transfer of money from one bank account to another requires two changes to the database both must succeed or fail together.

Process of Transaction

The transaction is executed as a series of reads and writes of database objects, which are explained below:

Read Operation

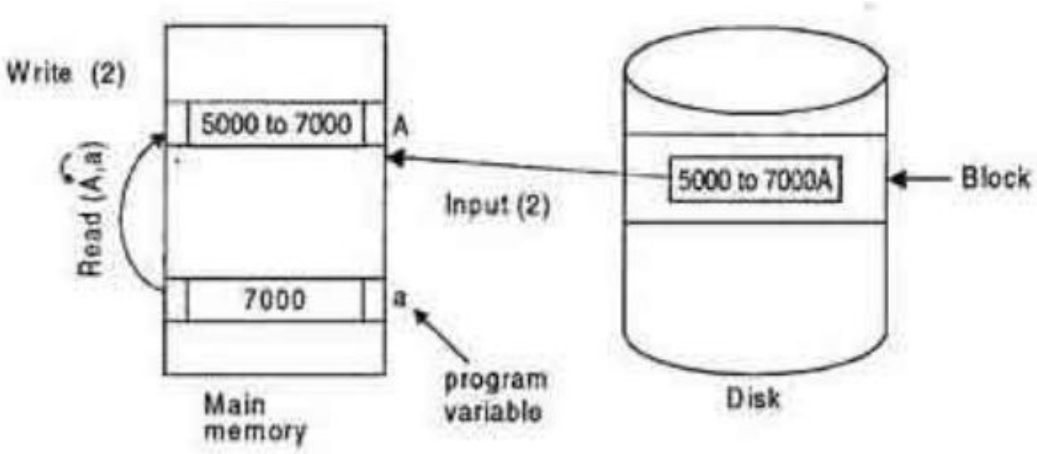
To read a database object, it is first brought into main memory from disk, and then its value is copied into a program variable as shown in figure.



Read Operation

Write Operation

To write a database object, an in-memory copy of the object is first modified and then written to disk.



Write Operation

Transaction Properties (ACID)

Atomicity: (all or nothing)

A transaction is said to be atomic if a transaction always executes all its actions in one step or not executes any actions at all It means either all or none of the transactions operations are performed.

Transaction Properties (ACID)

Consistency: (No violation of integrity constraints)

A transaction must preserve the consistency of a database after the execution.

The DBMS assumes that this property holds for each transaction. Ensuring this property of a transaction is the responsibility of the user.

Transaction Properties (ACID)

Isolation: (concurrent changes invisibles)

The transactions must behave as if they are executed in isolation.

It means that if several transactions are executed concurrently the results must be same as if they were executed serially in some order.

The data used during the execution of a transaction cannot be used by a second transaction until the first one is completed.

Transaction Properties (ACID)

Durability: (committed update persist)

The effect of completed or committed transactions should persist even after a crash. It means once a transaction commits, the system must guarantee that the result of its operations will never be lost, in spite of subsequent failures.

The acronym ACID is sometimes used to refer above four properties of transaction that we have presented here: Atomicity, Consistency, Isolation, and Durability.

Example

In order to understand above properties consider the following example:

Let, Ti is a transaction that transfers Rs 50 from account A to account B. This transaction can be defined as:

T1 Schedule
Read (A,a)
a:=a-50;
Write(A,a)
Read(B,b)
b:=b+50;
Write(B,b);

Atomicity

Suppose that, just prior to execution of transaction Ti the values of account A and B are Rs.I000 and Rs.2000.

Now, suppose that during the execution of Ti, a power failure has occurred that prevented the Ti to complete successfully. The point of failure may be after the completion Write (A,a) and before Write(B,b). It means that the changes in A are performed but not in B. Thus the values of account A and Bare Rs.950 and Rs.2000 respectively. We have lost Rs.50 as a result 'of this failure.

Now, our database is in inconsistent state.

The reason for this inconsistent state is that our transaction is completed partially and we save the changes of uncommitted transaction.

So, in order to get the consistent state, database must be restored to its original values i.e. A to Rs.I000 and B to Rs.2000, this leads to the concept of atomicity of transaction.

It means that in order to maintain the consistency of database, either all or none of transaction's operations are performed.

In order to maintain atomicity of transaction, the database system keeps track of the old values of any write and if the transaction does not complete its execution, the old values are restored to make it appear as the transaction never executed.

Consistency

The consistency requirement here is that the sum of A and B must be unchanged by the execution of the transaction.

Without the consistency requirement, money could be created or destroyed by the transaction.

It can. be verified easily that, if the database is consistent before an execution of the transaction, the database remains consistent after the execution of the transaction.

Ensuring consistency for an individual transaction is the responsibility of the application programmer who codes the transaction.

Isolation

If several transactions are executed concurrently (or in parallel), then each transaction must behave as if it was executed in isolation.

It means that concurrent execution does not result an inconsistent state.For example, consider another transaction T2, which has to display the sum of account A and B. Then, its result should be Rs.3000.

Isolation

Let's suppose that both Tl and T2 perform concurrently, their schedule is shown below:

T1	T2	Status
Read (A,a)		value of *A* i.e.1000 is copied to local variable a
a:=a-50		local variable *a*=950
Write(A,a)		value of local variable *a* 950 is copied to database item *A*
	Read(A,a)	value of database item *A* 950 is copied to *a*
	Read(B,b)	value of database item *B* 2000 is copied to *b*
	display(a+b)	2950 is displayed as sum of accounts *A* and *B*
Read(B,b)		value of data base item *B* 2000 is copied to local variable *b*
b:=b+50		local variable *b*=2050
Write(B,b)		value of local variable *b* 2050 is copied to database item *B*

The above schedule results inconsistency of database and it shows Rs.2950 as sum of accounts A and B instead of Rs.3000. The problem occurs because second concurrently running transaction T2, reads A and B at intermediate point and computes its sum, which results inconsistent value.

Isolation property demands that the data used during the execution of a transaction cannot be used by a second transaction until the first one is completed.

Isolation

A solution to the problem of concurrently executing transaction is to execute each transaction serially 'that is one after the other.

However, concurrent execution of transaction provides significant performance benefits, so other solutions are developed they allow multiple transactions to execute concurrently.

Durability

Once the execution of the transaction completes successfully, and the user who initiated the transaction has been notified that the transfer of funds has taken place, it must be the case that no system failure will result in a loss of data corresponding to this transfer of funds.

The durability property guarantees that, once a transaction completes successfully all the updates that it carried out on the database persist, even if there is a system failure after the transaction completes execution.

Ensuring durability is the responsibility of a component of the database system called the recovery-management component.

States of Transaction

A transaction must be in one of the following states:

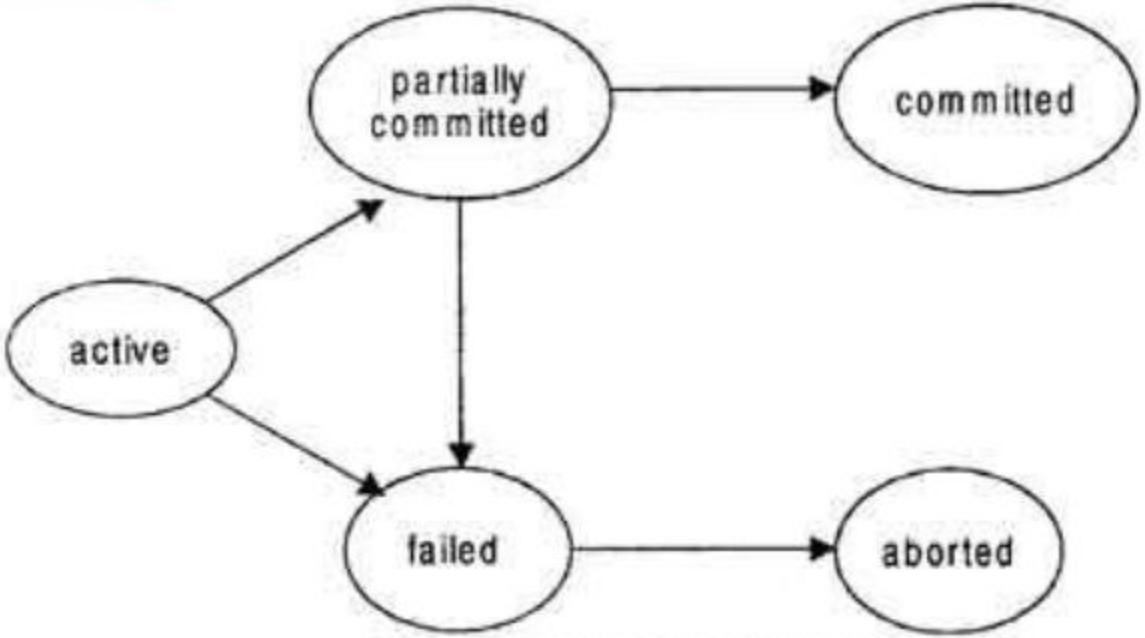
Active: the initial state, the transaction stays in this state while it is executing.

Partially committed: after the final statement has been executed.

Failed: when the normal execution can no longer proceed.

Aborted: after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.

Committed: after successful completion.



States of transaction

Advantages of Concurrent Execution of Transaction

The DBMS interleaves the actions of different transactions to improve performance of system as discussed below:

• Improved Throughput: Consider that transaction are performed in serial order and active transaction is waiting for a page to be read in from disk, then instead of CPU waiting for a page, it can process another transaction. This is because Input/Output activity can be done in parallel with the CPU activity. The overlapping of Input/Output activities of CPU reduces the amount of time disks and processors are idle and increases system throughput (the average number of transaction completed in a given time.)

• Reduced Waiting time: Interleaved execution of a short transaction with a long transaction usually allows the short transaction to complete quickly. In serial execution a short transaction could get stuck behind a long transaction leading to unpredictable delays in response time or average time taken to complete a transaction.

Transaction Isolation Levels in DBMS

The transaction isolation level is a state within databases that specifies the amount of data that is visible to a statement in a transaction, specifically when the same data source is accessed by multiple transactions simultaneously.

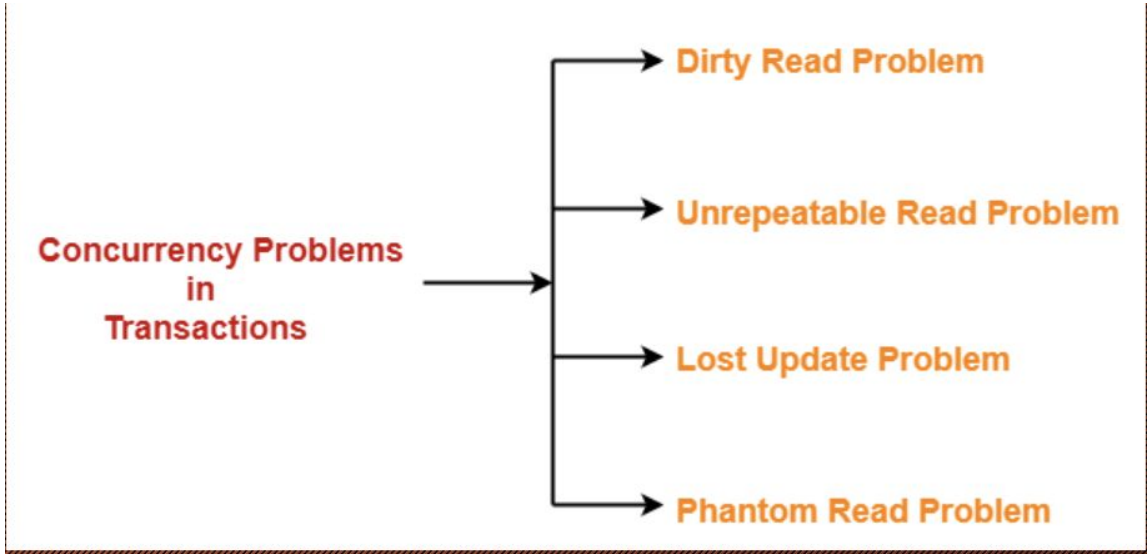
Transaction isolation level is part of the isolation state of a database management system. Isolation is one of the ACID (atomicity, consistency, isolation, durability) properties.

Isolation levels define the degree to which a transaction must be isolated from the data modifications made by any other transaction in the database system. A transaction isolation level is defined by the following phenomena –

Concurrency Problems in DBMS

When multiple transactions execute concurrently in an uncontrolled or unrestricted manner, then it might lead to several problems.

Such problems are called as concurrency problems.



Dirty Read – A Dirty read is the situation when a transaction reads a data that has not yet been committed. For example, Let’s say transaction 1 updates a row and leaves it uncommitted, meanwhile, Transaction 2 reads the updated row. If transaction 1 rolls back the change, transaction 2 will have read data that is considered never to have existed.

Non Repeatable read – Non Repeatable read occurs when a transaction reads same row twice, and get a different value each time. For example, suppose transaction T1 reads data. Due to concurrency, another transaction T2 updates the same data and commit, Now if transaction T1 rereads the same data, it will retrieve a different value.

Phantom Read – Phantom Read occurs when two same queries are executed, but the rows retrieved by the two, are different. For example, suppose transaction T1 retrieves a set of rows that satisfy some search criteria. Now, Transaction T2 generates some new rows that match the search criteria for transaction T1. If transaction T1 re-executes the statement that reads the rows, it gets a different set of rows this time.

Dirty Read Problem

Here,

T1 reads the value of A.

T1 updates the value of A in the buffer.

T2 reads the value of A from the buffer.

T2 writes the updated the value of A.

T2 commits.

T1 fails in later stages and rolls back.

Transaction T1	Transaction T2
R (A)	
W (A)	
⋮	R (A) // Dirty Read
	W (A)
	Commit
Failure	

Dirty Read Problem

In this example,

T2 reads the dirty value of A written by the uncommitted transaction T1.

T1 fails in later stages and roll backs.

Thus, the value that T2 read now stands to be incorrect.

Therefore, database becomes inconsistent.

Transaction T1	Transaction T2
R (A)	
W (A)	
⋮	R (A) // Dirty Read
	W (A)
	Commit
Failure	

Unrepeatable read problem

Non Repeatable read occurs when a transaction reads same row twice, and get a different value each time.

For example, suppose transaction T1 reads data.

Due to concurrency, another transaction T2 updates the same data and commit, Now if transaction T1 rereads the same data, it will retrieve a different value.

Unrepeatable read problem

Transaction T1	Transaction T2
R (X)	
	R (X)
W (X)	
	R (X) // Unrepeated Read

Unrepeatable read problem

Transaction T1	Transaction T2
R (X)	
	R (X)
W (X)	
	R (X) // Unrepeated Read

T1 reads the value of X (= 10 say).

T2 reads the value of X (= 10).

T1 updates the value of X (from 10 to 15 say) in the buffer.

T2 again reads the value of X (but = 15).

T2 gets to read a different value of X in its second reading.

T2 wonders how the value of X got changed because according to it, it is running in isolation.

Lost Update Problem

This problem occurs when multiple transactions execute concurrently and updates from one or more transactions get lost.(write-write conflict) blind write

Lost Update Problem

T1 reads the value of A (= 10 say).

T2 updates the value to A (= 15 say) in the buffer.

T2 does blind write A = 25 (write without read) in the buffer.

T2 commits.

When T1 commits, it writes A = 25 in the database.

Transaction T1	Transaction T2
R (A)	
W (A)	
⋮	W (A)
	Commit
Commit	

Phantom Read Problem

This problem occurs when a transaction reads some variable from the buffer and when it reads the same variable later, it finds that the variable does not exist.

Transaction T1	Transaction T2
R (X)	
	R (X)
Delete (X)	
	Read (X)

Phantom Read Problem

T1 reads X.

T2 reads X.

T1 deletes X.

T2 tries reading X but does not find it.

T2 finds that there does not exist any variable X when it tries reading X again.

T2 wonders who deleted the variable X because according to it, it is running in isolation

Transaction T1	Transaction T2
R (X)	
	R (X)
Delete (X)	
	Read (X)

Schedules

A sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed

A schedule for a set of transactions must consist of all instructions of those transactions

must preserve the order in which the instructions appear in each individual transaction.

A transaction that successfully completes its execution will have a commit instructions as the last statement

By default transaction assumed to execute commit instruction as its last step

A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

1. Serial Schedule

The serial schedule is a type of schedule where one transaction is executed completely before starting another transaction. In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.

For example: Suppose there are two transactions T1 and T2 which have some operations. If it has no interleaving of operations, then there are the following two possible outcomes:

Execute all the operations of T1 which was followed by all the operations of T2.

Execute all the operations of T2 which was followed by all the operations of T1.

In the given (1) figure, Schedule A shows the serial schedule where T1 followed by T2.

In the given (2) figure, Schedule B shows the serial schedule where T2 followed by T1.

1. Serial Schedule

T_1	T_2
read(A)	
$A := A - 50$	
write (A)	
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)

T_1	T_2
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	

2. Non Serial Schedule

If interleaving of operations is allowed, then there will be non-serial schedule.

It contains many possible orders in which the system can execute the individual operations of the transactions.

In the given figure (3) and (4), Schedule A and Schedule B are the non-serial schedules. It has interleaving of operations.

2. Non Serial Schedule

Let T1 and T2 be the transactions defined previously.

The following schedule is not a serial schedule, but it is equivalent to Schedule 1.

T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
read(B)	
$B := B + 50$	
write(B)	
	read(B)
	$B := B + temp$
	write(B)

2. Non Serial Schedule

The following concurrent schedule does not preserve the value of (A + B).

T_1	T_2
read(A)	
$A := A - 50$	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
write(A)	
read(B)	
$B := B + 50$	
write(B)	
	$B := B + temp$
	write(B)

3. Serializability

Basic Assumption – Each transaction preserves database consistency.

Serializability is a concurrency scheme where the concurrent transaction is equivalent to one that executes the transactions serially.

Serial schedule defines each transaction is executed consecutively without any interference from other transactions.

Non-serial schedule defines the operations from a group of concurrent transactions that are interleaved.

The main objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interference and produce a database state that could be produced by a serial execution.

Rules in Conflict Serializability

If two transactions are both read operation, then they are not in conflict.

Ordering of read/write is important

If one transaction wants to perform a read operation and other transaction wants to perform a write operation in same data item, then they are in conflict and cannot be swapped.

If one transaction wants to perform a read operation and other transaction wants to perform a write operation in separate item,they don’t conflict and order is not important

If both the transactions are for write operation, then they are in conflict, but can be allowed to take place in any order, because the transactions do not read the value updated by each other.

1. Conflict Serializability

A schedule is called conflict serializability if after swapping of non-conflicting operations, it can transform into a serial schedule.

The schedule will be a conflict serializable if it is conflict equivalent to a serial schedule.

Conflict serializability defines two instructions of two different transactions accessing the same data item to perform a read/write operation.

It deals with detecting the instructions that are conflicting in any way and specifying the order in which the instructions should execute in case there is any conflict.

A conflict serializability arises when one of the instruction is a write operation.

Conflicting Operations

Two operations are said to be conflicting if all conditions satisfy:

They belong to different transactions

They operate on the same data item

At Least one of them is a write operation

Instructions Ii and Ij of transactions Ti and Tj respectively, conflict if and only if there exists some item Q accessed by both Ii and Ij, and at least one of these instructions wrote Q.

Ii = read(Q), Ij = read(Q). Ii and Ij don't conflict.

Ii = read(Q), Ij = write(Q). They conflict.

Ii = write(Q), Ij = read(Q). They conflict

Ii = write(Q), Ij = write(Q). They conflict

1. T1: Read(A) T2: Read(A)

T1	T2
Read(A)	
	Read(A)

Swapped

T1	T2
	Read(A)
Read(A)	

Schedule S1 **Schedule S2**

2. T1: Read(A) T2: Write(A)

T1	T2
Read(A)	
	Write(A)

Swapped

T1	T2
	Write(A)
Read(A)	

Schedule S1 **Schedule S2**

Conflict Equivalent

In the conflict equivalent, one can be transformed to another by swapping non-conflicting operations. In the given example, S2 is conflict equivalent to S1 (S1 can be converted to S2 by swapping non-conflicting operations).

Two schedules are said to be conflict equivalent if and only if:

They contain the same set of the transaction.

If each pair of conflict operations are ordered in the same way.

Non-serial schedule

T1	T2
Read(A) Write(A)	
	Read(A) Write(A)
Read(B) Write(B)	
	Read(B) Write(B)

Schedule S1

Serial Schedule

T1	T2
Read(A) Write(A) Read(B) Write(B)	
	Read(A) Write(A) Read(B) Write(B)

Schedule S2

Schedule S2 is a serial schedule because, in this, all operations of T1 are performed before starting

any operation of T2. Schedule S1 can be transformed into a serial schedule by swapping non-conflicting

operations of S1.

After swapping of non-conflict operations, the schedule S1 becomes

T1	T2
Read(A) Write(A) Read(B) Write(B)	
	Read(A) Write(A) Read(B) Write(B)

Since, S1 is conflict serializable.

2. View Serializability

A schedule will view serializable if it is view equivalent to a serial schedule.

If a schedule is conflict serializable, then it will be view serializable.

The view serializable which is not conflict serializable contains blind writes.

It can be derived by creating another schedule out of an existing schedule and involves the same set of transactions.

View Equivalent

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:

1. Initial Read

An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

2. Updated Read

In schedule S1, if Ti is reading A which is updated by Tj then in S2 also, Ti should read A which is updated by Tj.

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

T1	T2	T3
Write(A)		
	Write(A)	
		Read(A)

Schedule S1

T1	T2	T3
	Write(A)	
Write(A)		
		Read(A)

Schedule S2

3. Final Write

A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

T1	T2	T3
Write(A)		
	Read(A)	
		Write(A)

Schedule S1

T1	T2	T3
	Read(A)	
Write(A)		
		Write(A)

Schedule S2

Concurrency Control

A database must provide a mechanism that will ensure that all possible schedules are

- either conflict or view serializable, and
- are recoverable and preferably cascadeless

■ A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency

- Are serial schedules recoverable/cascadeless?

■ Testing a schedule for serializability after it has executed is a little to late!

■Goal – to develop concurrency control protocols that will assure serializability.

Concurrency Control Protocol

Concurrency control protocols ensure atomicity, isolation, and serializability of concurrent transactions. The concurrency control protocol can be divided into two categories:

- Lock based protocol
- Time-stamp based protocol

Lock based protocol

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. a lock is a mechanism to control concurrent access to data item

Data items can be locked in 2 modes.

1. Shared lock:

It is also known as a Read-only lock. In a shared lock, the data item can only read by the transaction.

It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item.

2. Exclusive lock:

In the exclusive lock, the data item can be both reads as well as written by the transaction.

This lock is exclusive, and in this lock, multiple transactions do not modify the same data simultaneously.

Serial equivalence:

Serial equivalence requires that all of the transaction's accesses to a particular object be serialized with respect to accesses by other transactions.

To achieve serial equivalence transaction is not allowed any new locks after it has released a lock.

Two-phase lock: growing phase (new locks are acquired), shrinking phase (locks are released).

Strict execution: locks are held until transaction commits/aborts (Strict two-phase locking).

Recoverability: locks must be held until all the objects it updated have been written to permanent storage.

Simple exclusive lock reduces concurrency and locking schema for multiple transaction reading, single transaction writing i.e. read locks (shared lock) & write locks (exclusive lock).

The use of exclusive locks helps to serialize access to the objects.

The server attempts to lock any object that is about to be used by any operation of a client's transaction.

If client requests access to locked object, it must wait.

A transaction is not allowed any new locks after it has released a lock. The locks are applied in two phases:

First phase is growing phase and

Second phase is shrinking phase.

Operation conflict rules:

Request for a write lock is delayed by the presence of a read lock belonging to another transaction.

Request for either a read/write lock is delayed by the presence of a write lock belonging to another transaction.

For one object		Lock requested	
		Read	Write
Lock already set	none	OK	OK
	read	OK	wait
	write	Wait	wait

TRANSACTION T	TRANSACTION U

balance = b.getBalance( );
b.setBalance(balance*1.1);
a.withdraw(balance/10);
	balance = b.getBalance( );
	b.setBalance(balance*1.1);
	c.withdraw(balance/10);

openTransaction balance = b.getBalance( ); lock B b.setBalance(balance*1.1); a.withdraw(balance/10); lock A closeTransaction unlock A,B	openTransaction balance = b.getBalance( ); wait for T'lock on B … b.setBalance(balance*1.1); lock B c.withdraw(balance/10) ; lock C closeTransaction unlock B,C

Two Phase Locking (2PL)

The two-phase locking protocol divides the execution phase of the transaction into three parts.

In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires.

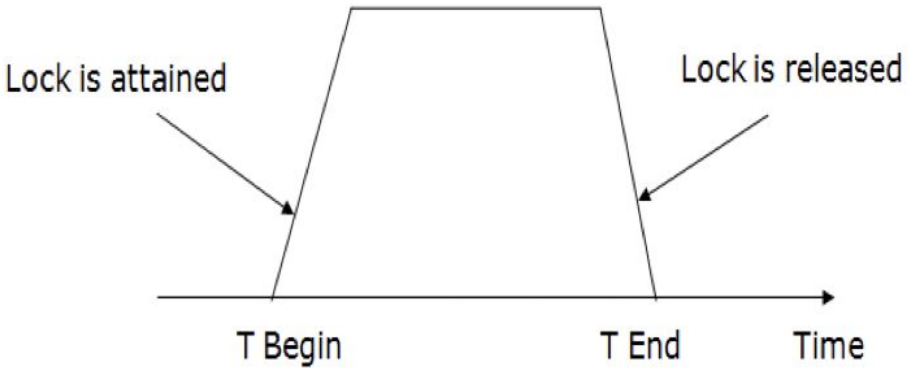
In the second part, the transaction acquires all the locks. The third phase is started as soon as the transaction releases its first lock.

In the third phase, the transaction cannot demand any new locks. It only releases the acquired locks.

Two Phase Locking (2PL)

Growing phase:
In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.

Shrinking phase:
In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired.



In the below example, if lock conversion is allowed then the following phase can happen:

Upgrading of lock (from S(a) to X (a)) is allowed in growing phase.

Downgrading of lock (from X(a) to S(a)) must be done in shrinking phase

The following way shows how unlocking and locking work with 2-PL.

Transaction T1:

Growing phase: from step 0-2

Shrinking phase: from step 4-6

Lock point: at 3

Transaction T2:

Growing phase: from step 1-5

Shrinking phase: from step 7-8

Lock point: at 6

	T1	T2
0	LOCK-S(A)	
1		LOCK-S(A)
2	LOCK-X(B)	
3	——	——
4	UNLOCK(A)	
5		LOCK-X(C)
6	UNLOCK(B)	
7		UNLOCK(A)
8		UNLOCK(C)
9	——	——

Time Stamp Ordering

The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps.

The order of transaction is nothing but the ascending order of the transaction creation.

The priority of the older transaction is higher that's why it executes first.

To determine the timestamp of the transaction, this protocol uses system time or logical counter.

The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time.

But Timestamp based protocols start working as soon as a transaction is created.

Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.

The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Time Stamp Ordering

- Each transactions' operation is validated when it is carried out.
- If operation cannot be validated, the transaction is aborted.
- Each transaction is assigned a unique timestamp value when it starts.
- **A transaction's request to write an object is valid only if that object was last read and written by earlier transactions.**
- A transaction's request to read an object is valid only if that object was last written by earlier transactions.

Time Stamp Ordering

Basic Timestamp ordering protocol works as follows:

Check the following condition whenever a transaction Ti issues a Read (X) operation:

If W_TS(X) >TS(Ti) then the operation is rejected.

If W_TS(X) <= TS(Ti) then the operation is executed.

Timestamps of all the data items are updated.

Where,

TS(TI) denotes the timestamp of the transaction Ti.

W_TS(X) denotes the Write time-stamp of data-item X.

Time Stamp Ordering

Check the following condition whenever a transaction Ti issues a Write(X) operation:

If TS(Ti) < R_TS(X) then the operation is rejected.

If TS(Ti) < W_TS(X) then the operation is rejected and Ti is rolled back otherwise the operation is executed.

Where,

TS(TI) denotes the timestamp of the transaction Ti.

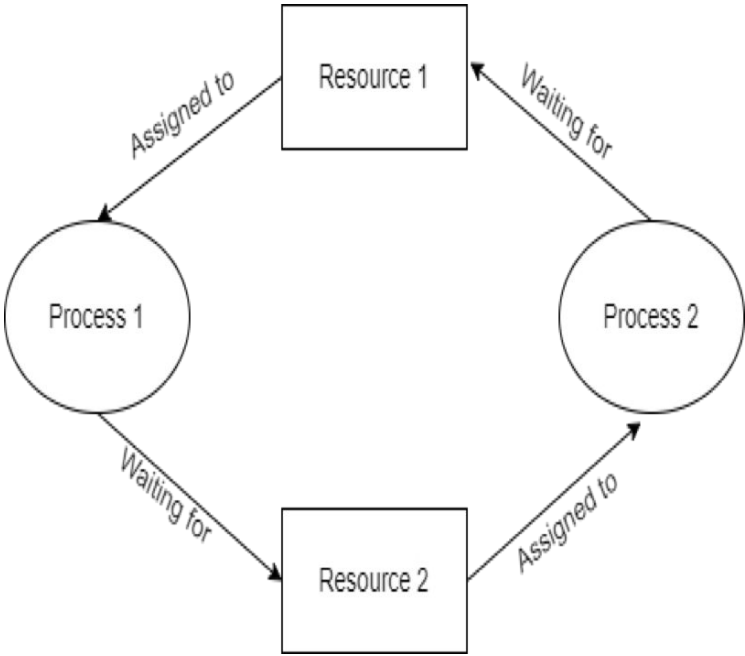
R_TS(X) denotes the Read time-stamp of data-item X.

W_TS(X) denotes the Write time-stamp of data-item X.

Deadlock

A Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource occupied by some other process.

When this situation arises, it is known as Deadlock.



Conditions for deadlock

The use of locks can lead to deadlock.

The situation in which two transactions are waiting and each is dependent on the other to release a lock so as to resume is called deadlock.

It can also be defined as a state in which each member of a group of transactions is waiting for some other member to release a lock.

Mutual exclusion: Each resource is either currently assigned to exactly one process, or is available.

Hold and wait: Processes hold resources already allocated to them while waiting for additional resources.

No preemption: Resources cannot be removed from the processes holding them until they are voluntarily released by processes holding them.

Circular wait: A circular chain of two or more processes, each of which is waiting for a resource held by the next process in the chain.